RELAYSPEC: RETARGETING FROZEN DRAFTERS FOR SPECULATIVE DECODING

Anonymous authors
Paper under double-blind review

ABSTRACT

A speculative drafter proposes several tokens that a larger target model checks together. Feature-conditioned drafters also use hidden states from the model they were trained with. Reusing such a drafter with a different target can therefore require running its original source transformer to supply those states. We introduce RelaySpec, which learns to supply the drafter's input from hidden states already computed by the new target. The drafter remains frozen. A bias-free linear map replaces the source transformer in the conditioning path, with normalization matched to the released drafter's interface. We evaluate this change with DFlash and EAGLE-3 drafters trained for Qwen3-4B and retargeted to Qwen3-8B and Qwen3-14B. On MATH-500, throughput increases by 1.08–1.48 times relative to optimized source reuse, with equal task accuracy in each paired comparison. The maps use 4,096 math problem-and-solution records and 52–66 million trainable weights. Separate EAGLE-3 measurements reduce peak allocated memory by 7.63–7.80 GiB. Across math, code and conversation workloads, component timings explain how the source computation saved and the tokens advanced per cycle jointly determine the gain. These results establish a practical use of learned feature maps for reusing frozen drafters across target model sizes.

1 INTRODUCTION

Speculative decoding lets a small model propose several tokens before a larger model checks them in one pass (Leviathan et al., 2023). We call the small model the *drafter* and the larger model the *target*. A good draft allows the target to commit several tokens per checking pass. The resulting speed depends on both the quality of those guesses and the time spent producing them.

DFlash and EAGLE-3 condition their guesses on internal vectors computed by a language model (Chen et al., 2026; Li et al., 2025). These vectors, called hidden states, contain the information the model has computed about a text prefix. A released drafter expects states from the particular model used during its training. We call that model its *source*. When a new target replaces the source, the source can still provide the drafter's features while the new target checks its proposals. This reuses the released drafter but adds source-model computation to generation.

We study a concrete deployment setting. An existing feature-conditioned drafter is held fixed, and a different target model is available. Can the target's own states supply the input the drafter needs? RelaySpec learns a matrix that maps selected target states into that input. The source and target provide paired feature vectors during fitting. At generation time, the map reads states from target verification and the source transformer is removed from the conditioning path. Figure 1 shows the two implementations compared in this paper.

The useful question is whether this substitution reduces total generation time. A map can produce less effective guesses than the original source features and still improve throughput if each cycle becomes sufficiently cheaper. We therefore measure throughput, task scores, tokens advanced per cycle and isolated memory. Feature-fitting error helps choose an interface, while generation measurements determine whether that choice is useful.

This paper makes three contributions. First, it specifies how to replace the conditioning path of two released drafter implementations while preserving their different normalization requirements. Second, it provides paired measurements of source reuse and RelaySpec for two Qwen3 target sizes, with

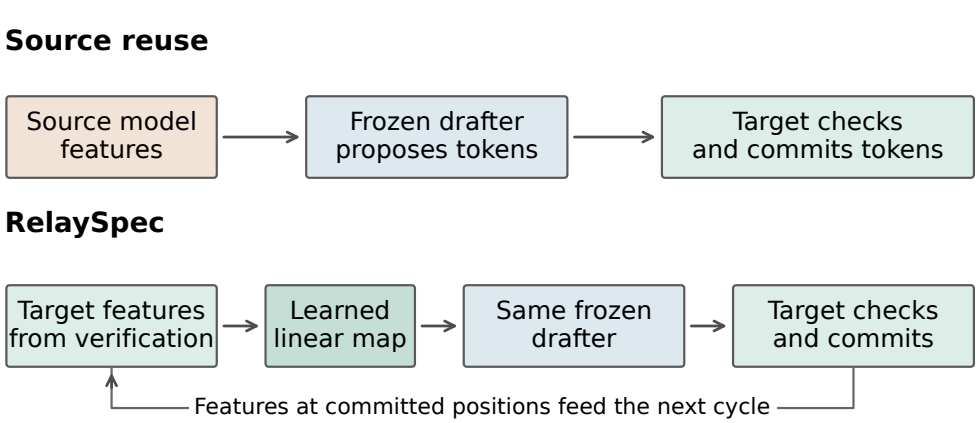

Figure 1: Replacing the computation that supplies a frozen drafter's input. Source reuse runs the original source transformer on the committed text. RelaySpec instead maps features from the new target into the same drafter interface. Both paths use the new target to check proposals. Source token embeddings and vocabulary projections used by the drafter remain fixed.

complete math, code and conversation results. Third, it relates the observed gains to source-model cost and accepted progress, and measures the memory reduction in separate deployment processes.

The comparison isolates the effect of replacing the source conditioning path. It holds the inherited drafter, target, decoding settings and request set fixed. This is a useful complement to training target-independent drafters (An et al., 2026) or steering pretrained drafters with target features (Berdoz et al., 2026). The method's distinction is the replacement of a released feature-conditioned drafter's source transformer with a fitted map, while retaining the drafter's weights.

2 Drafting with target verification

We first describe the greedy decoding procedure used in the experiments. Greedy decoding selects the token with the largest target score at each position. The drafter proposes a sequence of candidate tokens. The target then evaluates those candidates with a causal mask, so each position sees only the preceding text. The decoder keeps the longest candidate prefix that agrees with the target and appends the next token selected by the target. It discards the remaining candidate suffix.

For example, suppose the drafter proposes `the red car` and the target's choices agree through `the red` but then select `bus`. The decoder commits `the red bus`. The drafter saved checking rounds for the accepted prefix. It did not decide the correction. This example treats each displayed word as one token for clarity.

The target also stores intermediate attention states for the committed prefix. This stored state is its *cache*. After verification, the decoder crops the cache to the committed length and keeps the corresponding hidden states for the next draft. RelaySpec changes the vectors supplied to the drafter. The target's token-selection rule and cache bookkeeping remain part of the shared decoding implementation.

The standard greedy argument depends on a precise condition. Evaluating a candidate block must give the same target choice at each accepted position as evaluating that identical prefix one token at a time. With matching positions, masks, stopping rules and cache state, induction over committed tokens gives the same greedy sequence (Leviathan et al., 2023). This argument describes the verifier contract. Our empirical quality comparisons measure RelaySpec against source reuse in the same verifier implementation, using task scores and token-sequence agreement separately.

3 METHOD

3.1 READ TARGET FEATURES AT THE DRAFTER'S INPUT

We collect five hidden states at each text position and join them into one vector. Let h_t denote this vector from the new target at position t. Let s_t denote the corresponding vector from the source, and let F be the frozen projection supplied with the released drafter. The projection converts source states into the smaller context vector consumed by the drafter. We train a matrix W to produce that context from h_t.

The two drafter implementations consume different versions of this vector. For DFlash, let N_{in} be the fixed root-mean-square normalization of the joined target vector. It rescales that vector using the square root of its mean squared coordinate. Let N_{out} be the released drafter's frozen output normalization. The teacher vector c_t and predicted vector $\widehat{c}_t$ are

$$\begin{aligned} \text{DFlash:} \quad & c_t = N_{\text{out}}(Fs_t), \quad \widehat{c}_t = N_{\text{out}}(WN_{\text{in}}(h_t)), \\ \text{EAGLE-3:} \quad & c_t = Fs_t, \qquad\quad \widehat{c}_t = Wh_t. \end{aligned} \tag{1}$$

For EAGLE-3, Fs_t denotes the raw output of the released feature projection. Its scale is part of the consumed input. The selected relay therefore preserves target-feature scale. These choices follow the actual input used by each pinned implementation.

Only W is trained. It has no bias and no additional hidden layer. This keeps the added computation to one matrix multiplication and the fixed normalization where applicable. A linear map is a compact way to test whether the target already provides enough information for this input. Its suitability is an empirical question, answered here by matched interface checks and generation results.

3.2 FIT AGAINST SOURCE-DERIVED CONTEXT VECTORS

We run the frozen source and target on the same fitting text. For each position, we minimize the squared difference between the predicted vector and the source-derived teacher vector. We divide this difference by the teacher vector's squared length so that large-magnitude teachers do not automatically dominate the objective.

For a fitting record with n token positions, the loss is

$$\mathcal{L}(W) = \frac{1}{n} \sum_{t=1}^{n} \frac{\|\widehat{c}_t - c_t\|_2^2}{\max(\|c_t\|_2^2, \epsilon)}. \tag{2}$$

Here $\|v\|_2^2$ is the sum of the squared coordinates of vector v. The constant ϵ is the smallest positive normal value in 32-bit floating-point arithmetic. It protects division when the teacher is zero. We average positions within each record and then average the record losses across workers. Equation 1 determines which predicted and teacher vectors enter the loss.

This objective measures relative numerical error. It can penalize both scale and direction errors for the raw EAGLE-3 interface. For DFlash, both vectors pass through the released output normalization before comparison. That normalization also means the DFlash objective is not ordinary linear least squares in W. The experiments use gradient-based fitting for both families.

The fitting set contains 4,096 math problem-and-solution records. We render the problem and solution as conversation text and truncate the resulting input to 192 tokens. The optimization target is the source's internal context vector, rather than an answer-correctness label. Solution text is nevertheless part of the fitting input. The source, target and drafter are all frozen during this procedure.

3.3 USE THE MAP DURING GENERATION

Algorithm 1 describes the shared draft-and-check procedure. The map reads only hidden states from committed positions. Target verification does not read the mapped vectors. In the DFlash path, the frozen token embedding and output vocabulary projection used by the drafter are retained. Removing the source transformer refers specifically to the transformer layers that previously produced its context.

Algorithm 1 Generation with a frozen drafter and a learned feature map

```
Run the target on the prompt and retain its cache and selected states
Form drafter context using the appropriate map in Equation 1
while the stopping condition has not been reached do
    Generate candidate tokens with the frozen drafter
    Evaluate the candidates with the target and its causal mask
    Keep the longest prefix agreeing with the target's greedy choices
    Append the target-selected correction or next token, within the output cap
    Crop the cache and retained states to the committed positions
    Map the retained target states into context for the next draft
end while
```

4 WHY REMOVING THE SOURCE CAN SAVE TIME

Replacing source features has two effects. It removes source-transformer work, and it changes the number of tokens advanced by each drafting cycle. We use the recorded component times to explain their combined effect. A cycle is one proposal followed by one target verification.

Take the total source-reuse request time as one unit. Let p_S be the fraction spent running the source transformer. Let p_C be the fraction spent drafting and verifying candidate blocks. Let p_O cover prompt processing and other work. These fractions satisfy $p_S + p_C + p_O = 1$. Let a_S and a_R be the recorded mean tokens advanced per cycle for source reuse and RelaySpec. Let p_R be the relay's total mapping time, expressed in the same source-reuse time units.

For similar output lengths and approximately constant work per cycle, reducing progress per cycle increases the number of cycles by about a_S/a_R. If L_S and L_R denote total request time for source reuse and RelaySpec, respectively, this gives

$$\frac{L_R}{L_S} \approx \frac{a_S}{a_R} p_C + p_O + p_R. \tag{3}$$

This equation separates the work removed from the extra cycles induced by the mapped features. Within these assumptions, RelaySpec is faster when

$$\frac{a_R}{a_S} > \frac{p_C}{1 - p_O - p_R}, \qquad \text{provided } 1 - p_O - p_R > 0. \tag{4}$$

The left side is the fraction of original progress retained. The right side is the minimum fraction needed to offset the remaining work.

For intuition, suppose source processing consumes 30% of time, cycle-dependent work 65%, and other work 5%. If the map costs 1% and retains 80% of progress, Equation 3 gives a relative time of $0.65/0.80 + 0.05 + 0.01 = 0.8725$. This illustrative case saves about 13% of request time despite advancing fewer tokens per cycle. The calculation explains why accepted-token counts alone do not determine throughput.

Our analysis inserts timings and progress measured in the same completed runs. It is an accounting explanation of the observed operating range. It also makes the assumptions visible. Input processing, changing output lengths and different per-cycle costs can affect the approximation.

5 EXPERIMENTAL SETUP

Models and implementations. We use the released `z-lab/Qwen3-4B-DFlash-b16` drafter and `deepseek-ai/eagle3_qwen3_4b_ttt7`, an EAGLE-3 implementation from DeepSpec (Chen et al., 2026; Li et al., 2025; DeepSeek-AI, 2026). Both use Qwen3-4B as their source. Targets are Qwen3-8B and Qwen3-14B (Yang et al., 2025). DFlash proposes blocks of length 16. The EAGLE-3 implementation uses a draft length of 7. The comparison holds each implementation's proposal procedure fixed across conditioning methods.

Source-reuse comparison. The reference runs the source transformer only through its last required feature layer. It updates source features on the committed prefix and avoids executing the source on

Table 1: MATH-500 with 500 paired requests per row. Source denotes optimized source reuse and Relay denotes RelaySpec. The ratio compares their output tokens per second. Scores are answer accuracy percentages for source (S) and relay (R), computed from these runs.

Drafter	Target	Source tok/s	Relay tok/s	Ratio [95% interval]	Score S / R
DFlash	8B	148.33	219.99	1.483 [1.475, 1.492]	74.2 / 74.2
DFlash	14B	110.45	137.60	1.246 [1.238, 1.253]	79.6 / 79.6
EAGLE-3	8B	82.83	103.37	1.248 [1.242, 1.254]	73.2 / 73.2
EAGLE-3	14B	64.97	70.27	1.082 [1.076, 1.087]	79.6 / 79.6

rejected suffixes. RelaySpec uses the same frozen drafter and target, replacing this source path with the map. Each pair is measured on the same requests in the same execution, with method order rotated and warm-up requests excluded. This isolates the cost and behavior of changing the conditioning path.

Fitting and hardware. We use AdamW with learning rate 6×10^{-4}, zero weight decay, gradient clipping at 1.0 and 1,024 updates. Four workers each process one record per update, giving one presentation of each of the 4,096 records. Generation uses BF16 arithmetic, greedy choices, a 2,048-token output cap and disabled thinking mode. The reported experiments use four RTX 6000 Ada GPUs, each handling one request at a time. Timings describe the instrumented PyTorch/Transformers runtime. Appendix A specifies the feature layers, model widths and runtime versions.

Workloads and selection. We evaluate MATH-500, GSM8K, HumanEval, MBPP and MT-Bench (Hendrycks et al., 2021; Cobbe et al., 2021; Chen et al., 2021; Austin et al., 2021; Zheng et al., 2023). The main table contains all 500 MATH-500 questions. We also report the 468-question complement of the 32 questions used for development choices. This complement is computed from the same saved runs. The fitting and evaluation problem texts have zero normalized exact matches. MATH includes closely related problem templates, quantified in Appendix E. The fixed-map code and conversation evaluations provide a separate view of behavior across tasks.

Measurements. Throughput is total generated tokens divided by total measured request time, including prompt processing. The reported ratio divides RelaySpec throughput by source-reuse throughput. We report 95% intervals from 10,000 paired bootstrap resamples. Each resample draws whole matched requests, preserving their two method measurements. MT-Bench resamples whole two-turn conversations. Thus its 160 turns form 80 independent resampling units. These intervals describe request variation for the fixed fitted checkpoints.

We score math answers with the recorded Qwen math scorer and code with EvalPlus base and expanded test suites (Liu et al., 2023). A code score is the percentage of tasks whose single generated program passes the corresponding tests. MT-Bench contributes two-turn timing and token-agreement measurements. We report token-sequence agreement separately from task accuracy, because two different sequences can have the same answer or pass the same tests.

6 Results

6.1 Paired math throughput and task accuracy

Table 1 measures the effect of replacing source conditioning. On Qwen3-8B, DFlash throughput rises from 148.33 to 219.99 tokens per second. EAGLE-3 rises from 82.83 to 103.37 tokens per second. At Qwen3-14B, the corresponding changes are 110.45 to 137.60 and 64.97 to 70.27 tokens per second. Each of the four paired intervals lies above one.

Task accuracy is equal between source reuse and RelaySpec in each row. The recorded token sequences match on 500 of 500 DFlash requests at each target size and on 496 and 498 EAGLE-3 requests at 8B and 14B. The EAGLE-3 sequence differences preserve the recorded task outcomes. Mean output lengths and agreement counts are given in Appendix B. These measurements support the comparison between the two conditioning paths under the shared verifier.

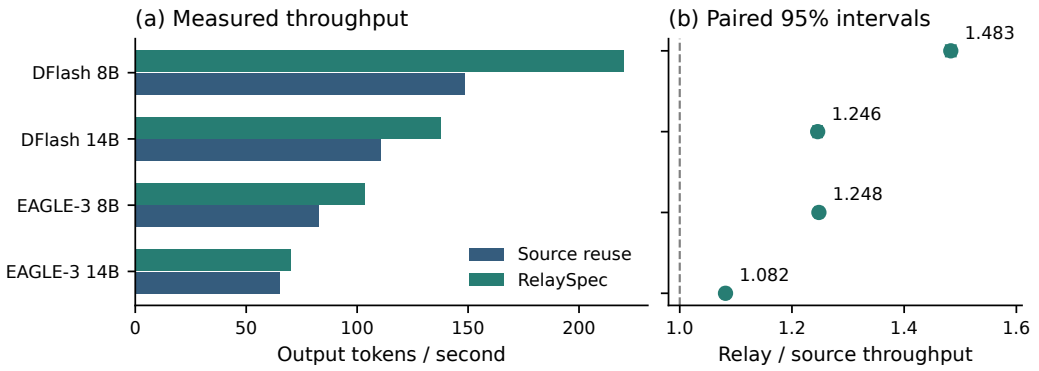

Figure 2: The same recorded MATH-500 results as Table 1. Absolute throughput gives the runtime scale. Paired intervals quantify the relative change when the source conditioning path is replaced.

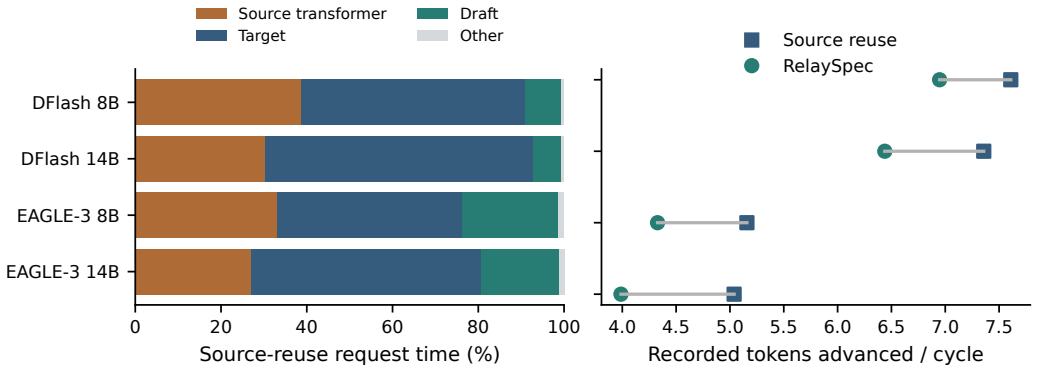

Figure 3: Left: component shares of source-reuse request time on MATH-500. Right: mean recorded tokens advanced per cycle, pooled over cycles. The map removes source-transformer work but also changes draft acceptance. Together these measurements explain the throughput changes.

The 468-question complement gives throughput ratios of 1.482 and 1.245 for DFlash, and 1.250 and 1.081 for EAGLE-3, at 8B and 14B respectively. The ordering remains the same after removing the development questions. Appendix B reports these intervals and scores separately from the full benchmark.

6.2 What changes inside a generation cycle?

Figure 3 separates source-reuse time into source processing, target processing, drafting and other work. It also compares the recorded progress per cycle. RelaySpec advances fewer tokens per cycle while spending less time on conditioning. The observed throughput gain is the net effect of both changes.

The gain is larger at 8B than at 14B for both drafter families. The same 4B source is a smaller share of the larger target's request time, so removing it provides less room to absorb additional verification cycles. This is consistent with the cost explanation in Equation 3. The runtime benefit comes from the work saved per generated token, rather than from improving the drafter's acceptance over source conditioning.

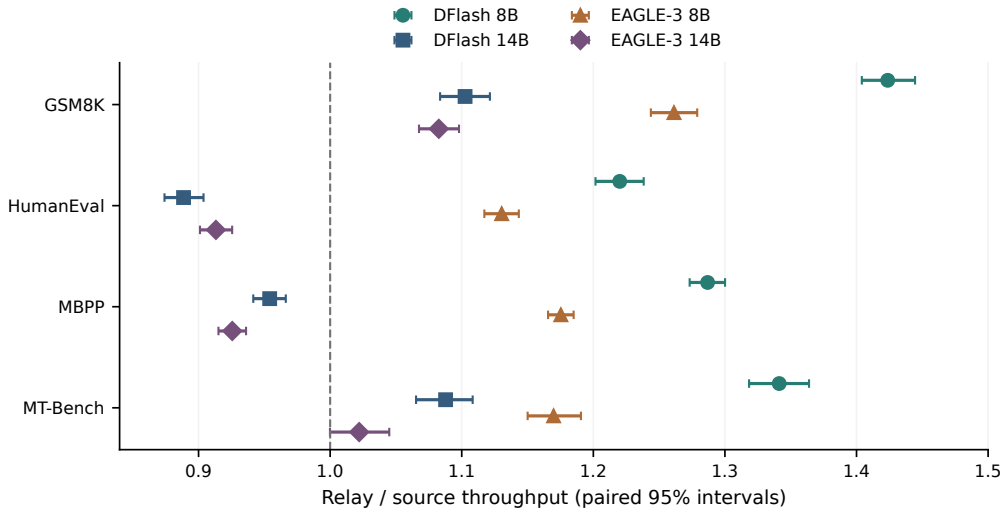

Figure 4: All four workloads with the fitted maps held fixed. A ratio above one favors RelaySpec and a ratio below one favors source reuse. Markers distinguish the four model/drafter settings in both color and print. Intervals resample requests, with MT-Bench clustered by conversation.

Table 2: Task scores in percent. Each entry is shared by source reuse and RelaySpec for that model/drafter pair. HumanEval+ and MBPP+ use the expanded EvalPlus tests on the same single generated program. MATH-500 scores are in Table 1.

Drafter	Target	GSM8K	HumanEval	HumanEval+	MBPP	MBPP+
DFlash	8B	93.75	85.98	80.49	84.13	73.02
DFlash	14B	94.53	89.02	83.54	88.36	75.13
EAGLE-3	8B	92.97	87.20	82.32	83.07	71.96
EAGLE-3	14B	94.53	90.24	85.98	88.89	74.87

6.3 One fitted map across tasks

We keep each math-fitted map fixed on GSM8K, HumanEval, MBPP and MT-Bench. Figure 4 shows all 16 drafter, target and task combinations. At 8B, both drafter families improve throughput on all four tasks. At 14B, GSM8K and DFlash MT-Bench retain clear gains. The 14B coding ratios range from 0.889 to 0.954, and the EAGLE-3 MT-Bench interval includes one. The complete matrix identifies the workloads where source removal outweighs the extra cycles, rather than treating acceptance as a speed measure.

The relay retains 61.5–85.4% of source-reuse progress per cycle across these cells. The source transformer consumes 26.8–40.0% of reference request time, while mapping costs 0.38–0.72% in the same units. Substituting each run's measured components and progress into Equation 3 reconstructs the observed direction in all 16 cells, with 0.34% mean absolute relative error. This close agreement summarizes the recorded cost balance within these measurements.

Table 2 reports the scored workloads. Source reuse and RelaySpec have the same recorded task outcomes across the 4,680 paired math and code examples, counting each problem once per model/drafter pair. Base and expanded code tests are two scoring views of the same generated program. The throughput results therefore measure different execution costs for equal observed task outcomes in these comparisons.

6.4 Fitting cost and deployed memory

Each 8B map has 52.43 million trainable weights and each 14B map has 65.54 million. Table 3 reports the recorded fitting loop duration. These timings begin with models loaded and cover feature

Table 3: Map size and recorded fitting-loop time on four GPUs. Every row uses 4,096 distinct fitting records and 1,024 updates. The timed boundary covers the training loop with models already loaded.

Drafter	Target	Trainable weights (millions)	Fitting loop (seconds)
DFlash	8B	52.43	85.6
DFlash	14B	65.54	118.0
EAGLE-3	8B	52.43	86.7
EAGLE-3	14B	65.54	118.5

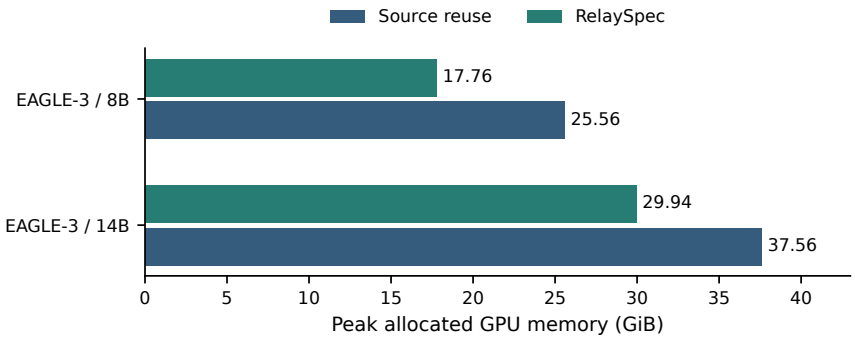

Figure 5: Peak allocated GPU memory from separate EAGLE-3 processes with 8 paired prompts per target. The same target and drafter are used in each comparison. Source removal saves 7.80 GiB at 8B and 7.63 GiB at 14B.

extraction and optimization within that loop. They measure this stage of adaptation, separately from model loading, checkpoint writing and deployment setup. The inherited drafter's original training is also a separate cost.

To measure memory after source removal, we run source reuse and RelaySpec in separate processes with identical EAGLE-3 targets, prompts and settings. Peak allocated memory falls from 25.56 to 17.76 GiB at 8B and from 37.56 to 29.94 GiB at 14B. A GiB is 2^{30} bytes of memory. These reductions are 30.5% and 20.3%, respectively. Figure 5 shows these isolated measurements. Mixed-method timing processes are used for paired speed comparisons, while these separate processes measure deployment memory.

6.5 Match the map to the consumed interface

A controlled EAGLE-3 development comparison changes only whether target features are normalized before mapping. Preserving their scale reduces relative fitting error from 0.413 to 0.266 after 128 updates. On the same 32 development prompts, the throughput ratio over source reuse rises from 1.035 to 1.237. The parameter count and fitting budget are held fixed. This supports preserving scale at the raw EAGLE-3 input.

For DFlash, we compare relative feature error with a squared-error plus cosine-distance objective at the same fitting budget. Direct paired comparisons between the fitted candidates favor relative error by factors of 1.009 at 8B and 1.011 at 14B. This is a small measured improvement and removes the additional cosine coefficient from the selected recipe. Appendix D gives the development results and their intervals.

7 Related work

Training drafters and reducing drafting work. Speculative decoding separates candidate generation from target verification (Leviathan et al., 2023). Medusa adds multiple prediction heads (Cai et al., 2024). EAGLE-3 trains a drafter using multi-layer features and a training procedure that exposes it to its own predictions (Li et al., 2025). DFlash generates blocks through a diffusion

drafter (Chen et al., 2026). RepSpec expands linear components during training and merges them for inference (Huo et al., 2026). RelaySpec acts after drafter training. It learns to provide a released feature-conditioned drafter with context from a new target.

Reusing drafters across targets. PARD trains a parallel drafter that can serve multiple targets without separate target-specific adaptation (An et al., 2026). SD^2 injects target information into pretrained drafters and studies both frozen and fine-tuned drafter variants (Berdoz et al., 2026). OmniDraft combines cross-vocabulary drafting with online adaptation (Ramakrishnan et al., 2025). Algorithms for heterogeneous vocabularies also establish how proposals can be checked across tokenizer boundaries (Timor et al., 2025). These methods address related reuse settings. Our experiments isolate a different operation: removing the source transformer from a released feature-conditioned drafter's input path while fitting only a map for a fixed target.

Connecting frozen models. Learning a connector between frozen networks is established in model stitching (Bansal et al., 2021). RelaySpec applies that idea to the context consumed inside a speculative decoding loop. We evaluate the connection through generated-token progress, runtime and task outcomes. This interpretation is specific to the measured behavior. Successful connections alone do not establish that two models encode the same information (Smith et al., 2025).

Measuring inference improvements. The time spent drafting, verifying and managing requests depends on the runtime and hardware. SPEED-Bench studies this variation across speculative decoding implementations (Abramovich et al., 2026). We therefore use matched executions for source reuse and RelaySpec, report absolute throughput alongside ratios, and measure source-removal memory separately. Published results from other runtimes provide context for the deployment problem rather than a common numerical ranking.

8 Scope and conclusion

The empirical claim concerns retargeting two fixed Qwen3-4B drafters to 8B and 14B targets under greedy, single-request BF16 execution. The reported task-quality comparison is against source reuse in the same verifier implementation. Request bootstrap intervals condition on the selected fitted checkpoint. Fitting data come from math, and the full workload matrix shows how the result varies across tasks. This scope makes the measured advantage and its operating conditions explicit.

RelaySpec shows that target states can supply the context used by a frozen feature-conditioned drafter. A learned map replaces source-transformer execution while retaining the drafter's weights. Paired measurements show the resulting throughput change, task outcomes and memory reduction. The governing balance is concrete: the source computation saved must outweigh the extra drafting cycles caused by imperfect context matching. Matching the consumed interface provides a simple way to make this substitution useful.

AI use statement

Generative AI tools assisted with hypothesis development, implementation, code review, literature search, analysis scripts, and manuscript drafting and editing. Numerical tables and figures in this version are generated from recorded experimental artifacts. The accompanying checks compare saved outputs, request counts, task scores and timings with their source records. Scientific interpretation and submission decisions remain the authors' responsibility.

Reproducibility statement

Appendix A gives the fitting and generation settings. Appendices B and C give complete task results. Appendices D and E describe development choices and data checks. The accompanying implementation contains the model identifiers, configurations, saved outputs and scripts used to rebuild the tables and figures. The generated evidence registry links reported workload cells to their underlying request records.

REFERENCES

Talor Abramovich, Maor Ashkenazi, Carl Putterman, Benjamin Chislett, Tiyasa Mitra, Bita Darvish Rouhani, Ran Zilberstein, and Yonatan Geifman. SPEED-Bench: A unified and diverse benchmark for speculative decoding. In *43rd International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2604.09557. Accepted paper.

Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. PARD: Accelerating LLM inference with low-cost PARallel draft model adaptation. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=XbOyv7iVGL.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL https://arxiv.org/abs/2108.07732.

Yamini Bansal, Preetum Nakkiran, and Boaz Barak. Revisiting model stitching to compare neural representations. In *Advances in Neural Information Processing Systems 34*, 2021. URL https://papers.nips.cc/paper/2021/hash/01ded4259d101feb739b06c399e9cd9c-Abstract.html.

Frédéric Berdoz, Peer Rheinboldt, and Roger Wattenhofer. Steering pretrained drafters during speculative decoding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 30067–30075, 2026. doi: 10.1609/aaai.v40i36.40255. URL https://ojs.aaai.org/index.php/AAAI/article/view/40255.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL https://proceedings.mlr.press/v235/cai24b.html.

Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. In *43rd International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2602.06036. Accepted paper.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL https://arxiv.org/abs/2107.03374.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

DeepSeek-AI. DeepSpec: A full-stack codebase for training and evaluating speculative decoding draft models. Official open-source software, 2026. URL https://github.com/deepseek-ai/DeepSpec.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems 34, Datasets and Benchmarks Track*, 2021. URL https://openreview.net/forum?id=7Bywt2mQsCe.

Feiye Huo, Jianchao Tan, Jiahao Liu, Zixu Jiang, Jiacheng Li, Jingang Wang, Xunliang Cai, and Shengli Sun. RepSpec: Structural re-parameterized draft model training for speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/d70ea003729b440b89a2f958a5554c1f-Abstract-Conference.html.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 19274–19286, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems 38*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/c7b5a35ea98b62512a869c19ea7b03cb-Abstract-Conference.html.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html.

Ramchalam Kinattinkara Ramakrishnan, Zhaocong Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chenzheng Su, and Xiaopeng Zhang. OmniDraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding. In *Advances in Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3c2fe1417eed1c6ff9acf169617981ea-Abstract-Conference.html.

Damian Smith, Harvey Mannering, and Antonia Marcu. Functional alignment can mislead: Examining model stitching. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 55972–55998, 2025. URL https://proceedings.mlr.press/v267/smith25a.html.

Nadav Timor, Jonathan Mamou, Daniel Korat, Moshe Berchansky, Gaurav Jain, Oren Pereg, Moshe Wasserblat, and David Harel. Accelerating LLM inference with lossless speculative decoding algorithms for heterogeneous vocabularies. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL https://proceedings.mlr.press/v267/timor25a.html.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.

A Implementation and measurement details

Feature layers and matrix dimensions. Layer indices count transformer blocks from zero. We take the output states of blocks $[1, 9, 17, 25, 33]$ from the source and from Qwen3-8B. For Qwen3-14B we use $[1, 10, 19, 28, 37]$. Qwen3-8B has a hidden width of 4,096, so joining five states gives 20,480 coordinates. Qwen3-14B has a hidden width of 5,120 and gives 25,600 coordinates. The drafter interface has width 2,560. Thus W has shape $2,560 \times 20,480$ at 8B and $2,560 \times 25,600$ at 14B. Multiplying the two dimensions gives 52,428,800 and 65,536,000 weights. The DFlash input normalization has no trainable gain or bias. The drafter's output normalization is frozen.

Teacher construction and optimization. The source processes the same rendered fitting text as the target. DFlash supervision is the released fusion output after the drafter's normalization. EAGLE-3 supervision is the released raw feature projection. We compute the loss in 32-bit arithmetic and use BF16 model computation. Fitting uses random seed 1729. The four workers average gradients after each record, giving four record losses per update. The AdamW learning rate is 0.0006, weight decay is zero, and the gradient norm is clipped at 1.0. Fitting uses 1,024 updates and a maximum rendered length of 192 tokens. Every main result uses the selected map for that drafter and target pair.

Source reuse. The optimized source provider retains a cache for the committed sequence and executes only the newly committed tokens through the required source layers. It omits source layers beyond block 33. The relay provider instead reads the target outputs retained from verification. Both providers hand context to the same downstream drafter. Rejected target-cache suffixes are cropped before the next iteration. The DFlash path retains the frozen source embedding and vocabulary projection used for token drafting.

Runtime. Each worker serves a request sequentially on one RTX 6000 Ada GPU. The four GPUs distribute requests rather than splitting one request across devices. The recorded runtime uses Python 3.12.13, PyTorch 2.9.1 with CUDA 12.8, and Transformers 5.3.0 for DFlash or 5.10.2 for EAGLE-3. The configuration uses dense scaled dot-product attention in PyTorch/Transformers. Requests are synchronized before and after timing. Timed work includes target prompt processing, drafting, verification and conditioning updates. Model loading and task scoring occur outside this request timer. One warm-up per method precedes the timed requests, and method order rotates across requests. Each recorded request has one measurement per method.

Stopping and conversation history. Generation stops at the model's end token or at the 2,048-token cap. Capped requests remain in the timing and scoring aggregates. Each MT-Bench method uses its own first-turn answer as context for its second turn. The complete two-turn EAGLE-3 runs supply the reported conversation cells. The two turns of one conversation form one bootstrap unit.

How throughput is aggregated. For each method, add the generated-token counts over all measured requests and divide by their summed request times. Dividing the relay value by the source-reuse value gives the reported ratio. This definition accounts for the small observed output-length differences. It is not the mean of per-request ratios. Each paired bootstrap resample recomputes this same aggregate from resampled whole requests or conversations. The 95% interval is the central range of 10,000 such resamples, using bootstrap seed 1729.

B Math scores, agreement and development complement

Table 4 separates exact token-sequence agreement from task accuracy. Agreement compares the recorded hashes of generated token IDs. The scorer compares the extracted answer with the dataset reference. Both checks refer to source reuse and the relay within a paired run.

The 32 development questions influenced interface and objective choices. Table 5 therefore also reports the other 468 questions, using the saved outputs of the full run. It is a subset analysis of these experiments, with its exposure history preserved. The full table and complement answer different reporting questions and are shown separately.

Table 4: MATH-500 token agreement and mean output length. S denotes source reuse and R denotes RelaySpec. All 500 requests contribute to each row.

Drafter	Target	Matching token sequences	Mean tokens S	Mean tokens R
DFlash	8B	500/500	780.29	780.29
DFlash	14B	500/500	775.45	775.45
EAGLE-3	8B	496/500	775.16	775.20
EAGLE-3	14B	498/500	766.12	766.14

Table 5: The 468-question MATH-500 complement after removing development questions. Ratios compare relay and source-reuse throughput. Each score is shared by the two methods and is expressed as a percentage.

Drafter	Target	Ratio [95% interval]	Score S = R
DFlash	8B	1.482 [1.473, 1.490]	73.93
DFlash	14B	1.245 [1.237, 1.253]	79.27
EAGLE-3	8B	1.250 [1.244, 1.256]	72.86
EAGLE-3	14B	1.081 [1.076, 1.087]	79.49

C Complete fixed-map workload results

Table 6 includes every cell in the four-task evaluation. The map for a given target and drafter stays fixed across these tasks. GSM8K has 128 requests, HumanEval has 164, and MBPP has 378. MT-Bench has 160 turns from 80 conversations. Code scoring uses one generated program per task with both the base and expanded EvalPlus tests. Table 2 reports both scoring views.

Table 6: Complete source-reuse and relay throughput matrix. All intervals use paired resampling. MT-Bench resamples 80 conversations, each with two turns. Values below one identify settings where source reuse is faster.

Drafter	Target	Task	Requests	Source tok/s	Relay tok/s	Ratio [95% interval]
DFlash	8B	GSM8K	128	116.87	166.39	1.4237 [1.4040, 1.4445]
DFlash	8B	HumanEval	164	118.84	145.00	1.2201 [1.2017, 1.2383]
DFlash	8B	MBPP	378	117.77	151.55	1.2868 [1.2732, 1.3001]
DFlash	8B	MT-Bench	160	60.85	81.62	1.3412 [1.3183, 1.3639]
DFlash	14B	GSM8K	128	88.04	97.07	1.1025 [1.0836, 1.1214]
DFlash	14B	HumanEval	164	87.34	77.61	0.8886 [0.8741, 0.9038]
DFlash	14B	MBPP	378	88.02	83.98	0.9541 [0.9416, 0.9663]
DFlash	14B	MT-Bench	160	45.32	49.30	1.0877 [1.0653, 1.1083]
EAGLE-3	8B	GSM8K	128	85.99	108.45	1.2613 [1.2437, 1.2789]
EAGLE-3	8B	HumanEval	164	71.75	81.10	1.1304 [1.1172, 1.1434]
EAGLE-3	8B	MBPP	378	69.85	82.10	1.1753 [1.1656, 1.1850]
EAGLE-3	8B	MT-Bench	160	42.27	49.45	1.1698 [1.1501, 1.1906]
EAGLE-3	14B	GSM8K	128	67.81	73.41	1.0826 [1.0675, 1.0979]
EAGLE-3	14B	HumanEval	164	55.38	50.58	0.9132 [0.9011, 0.9255]
EAGLE-3	14B	MBPP	378	54.55	50.49	0.9256 [0.9151, 0.9361]
EAGLE-3	14B	MT-Bench	160	33.83	34.57	1.0221 [0.9999, 1.0449]

The pooled math-and-code count is four model/drafter pairs times $500 + 128 + 164 + 378 = 1,170$ problems, giving 4,680 paired outcomes. Conversation turns are outside this scored total. Expanded code tests reuse the same generated programs and do not add independent examples. Equal observed outcomes describe these records and the stated scorers.

D Development comparisons

Preserving EAGLE-3 input scale. We fit two maps with the same source, target, parameter count, optimizer and 128-update budget. One normalizes the joined target features before the linear map.

The other uses the raw features. We then evaluate both on the same 32 development questions. Table 7 reports the recorded result. Raw features retain magnitude information that the released EAGLE-3 interface can consume.

Table 7: Matched EAGLE-3 development comparison on Qwen3-8B. Relative error uses Equation 2. Progress retained compares recorded tokens advanced per cycle with source reuse.

Input to linear map	Relative error	Throughput ratio [95% interval]	Progress retained
Normalized features	0.413	1.035 [1.003, 1.068]	69.0%
Raw features	0.266	1.237 [1.213, 1.263]	82.6%

DFlash fitting objective. We compare the selected relative-error objective against mean squared error plus 0.1 times cosine distance. Cosine distance measures how strongly two vectors differ in direction. Both candidates use 4,096 records, 1,024 updates, the same normalization and the same optimizer settings. Table 8 reports their separate source-reuse comparisons. Directly comparing the two fitted candidates gives ratios of 1.0085 [1.0005, 1.0174] at 8B and 1.0114 [1.0046, 1.0177] at 14B.

Table 8: DFlash objective selection on 32 development questions. Each throughput ratio is relative to that candidate's paired source-reuse arm. The direct candidate comparison is reported in the text.

Target	Objective	Throughput ratio [95% interval]	Token matches
8B	Relative error	1.523 [1.488, 1.558]	32/32
8B	Squared error + cosine	1.509 [1.476, 1.545]	32/32
14B	Relative error	1.265 [1.243, 1.289]	32/32
14B	Squared error + cosine	1.251 [1.229, 1.276]	32/32

Progress within a cycle. Figure 6 plots the fraction of recorded cycles that reach each position. Summing these fractions gives the pooled mean progress per cycle. It weights cycles equally, whereas averaging a mean for each request gives requests equal weight. Our cost analysis uses the pooled cycle quantity consistently. The recorded lengths include the implementation's target-supplied progress as well as accepted draft positions. They are not probabilities that arbitrary drafter tokens are correct independently of their prefix.

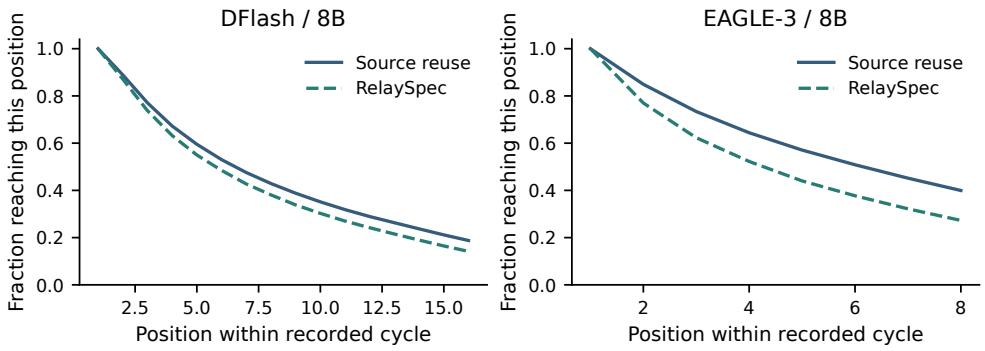

Figure 6: Fraction of recorded cycles reaching each position on MATH-500 for the two 8B settings. Solid lines denote source reuse and dashed lines denote RelaySpec.

E Data checks and fitting text

The fitting manifest contains 4,096 distinct problem-and-solution records. The loader renders both fields and applies the 192-token cap afterward. One fitting pass therefore means 4,096 record

presentations, with text length determined by the rendered cap. Repeating that manifest changes the number of presentations rather than the number of distinct records.

We compare normalized problem text between fitting and evaluation manifests. The exact-match audit finds zero matches among the 4,096 fitting records and 1,250 evaluation records. MT-Bench contributes 80 conversation records to this manifest count. A separate similarity check splits each text into unique normalized tokens and computes the size of their intersection divided by the size of their union. This is token-set Jaccard overlap. It measures shared text tokens, rather than character alignment or mathematical equivalence.

Table 9 reports the maximum fitting overlap for each evaluation task and the number of evaluation records above three thresholds. In MATH-500, 47 problems exceed 0.80 and four exceed 0.95. These similarities qualify the interpretation of the math fitting and evaluation relationship. The code and conversation measurements use the same fitted maps and provide additional task coverage.

Table 9: Fitting-to-evaluation token-set overlap. Each evaluation record is compared with its closest fitting record. The final three columns count evaluation records at or above the stated overlap threshold.

Task	Maximum overlap	≥ 0.80	≥ 0.90	≥ 0.95
MATH-500	0.980	47	13	4
GSM8K	0.378	0	0	0
HumanEval	0.290	0	0	0
MBPP	0.333	0	0	0
MT-Bench	0.571	0	0	0

F INTERPRETING MEMORY AND FITTING TIME

For each isolated memory comparison, source reuse and RelaySpec start in separate processes with the same target, drafter, prompts and generation settings. We reset GPU peak counters before the measured requests and report peak allocated memory. These values count active tensor allocations. Reserved allocator memory is a distinct measurement. The two EAGLE-3 target settings each use eight paired prompts.

The fitting-loop timer covers source and target feature computation and matrix optimization with the models already loaded. Its boundary is the same across the reported fitting summaries. A complete setup also includes loading weights, writing the fitted map and initializing generation. Separating these operations makes the recorded loop time interpretable without equating it with a complete deployment time.